P2.9 — deployment plan

Status: steps 1 to 7 done, 7 October 2026 (the gate passed 5 of 5); step 8, the demo recording, after P2.11 (note of 10 October in step 8) **Predecessor:** P2.9b, closed 29 September 2026 (`docs/p2_9b_closure.md`) **Successor:** Phase 2 checkpoint, then the API, then the MCP server

---

1. What P2.9 is

P2.9 puts the app on Streamlit Community Cloud from a public repository, and proves it works there for somebody who is not the author.

It is **not** a feature phase. Nothing in this plan changes behaviour. Everything here is verification, hygiene, and one irreversible publishing step.

Definition of done: a deployed app that renders a real network and lets a visitor correct a grey node. That is also the Phase 2 checkpoint.

2. The eight steps, in order

The order is not arbitrary — steps 1 to 3 are all cheaper to fix before the repo is public than after.

Step 1 — Clean-venv manifest install (done, 6 October)

Verify `requirements.txt` resolves on Python 3.11 and the app boots from nothing but the lock. Run it with a fresh interpreter outside the repo, so the working `.venv` is never on PATH.

Passes when: - `pip install -r requirements.txt` succeeds and `pip check` is clean - `tools.verify_manifests` and `tools.import_closure` both pass - the app boots with no `ANTHROPIC_API_KEY` and lands on the demo - `pyarrow` is **24.0.0** (D-52)

Three expected absences, each a pass not a failure: - `pytest` does not install — it is dev-only, and a clean venv that can run the suite means the manifest split has leaked - `matplotlib` does not install — Decision B moved it to dev-only via the lazy import inside `graph.plot_graph` - nothing from `requirements-dev.in` appears

This is the step most likely to surprise, and the cheapest to run.

Result, 6 October: passed on the first run. - `tools.verify_manifests`: 10/10. All three expected absences held: no `pytest`, no `matplotlib`, nothing from `requirements-dev.in`. - `pip check` was clean. `pyarrow` is 24.0.0, pinned in the lock (`requirements.txt`, D-52). - The app booted with no `ANTHROPIC_API_KEY` and landed on the demo. - `tools.import_closure` was not run separately. `verify_manifests` already imports every build-chain and dashboard module in a runtime-only venv and fails if any dev-only package is installed or loaded, which is the boundary `import_closure` was listed to check. An `import_closure --all` run, suggested earlier as a correction to a wrong command, would add nothing to that.

What it produced besides a pass. The step itself passed. Everything else was incidental, from pulling a thread it exposed. A keyed build of the author's export through the upload path, run while verifying the install, left the tracked classification cache modified. That led to: - **D-71**: session titles were being written to the shared cache. - **D-72**: the five-minute build has no progress and no recovery, measured on the same run. - **D-73**: duplicate normalised keys, found verifying the first D-71 commit. - **D-74**: four tests were making real API calls, found verifying the D-71 fix.

D-71 and D-74 are closed; D-72 and D-73 are open. D-75 came from the same thread by way of step 2, and is recorded there.

Step 2 — README pass (done, 6 October)

- **O*NET attribution**: one attribution, escaped as `O\*NET`, under `## Data attribution` and before `## Development`. The duplicate was removed on 29 September (`95a9dbc`); it is confirmed unchanged. A prose mention at line 50 is unescaped, but its paragraph has no second asterisk, so it cannot open emphasis.
- **Figures are consistent after D-69.** They were already correct. README lines 54–61 carry the spread, the mean and the 15% title-level disagreement, and state that the layout is seeded while the classification is not. No README sentence says the demo understates the real 29.0% "by about 7 points". The 21.9% figure belonged to the demo shipped on 24 September. The demo now ships at 124 of 442 (28.05%), run 5 of the five draws. Every classification figure in the project is a single-run point estimate.
- **Caches**: rewritten for D-71 (`197102b`). Both caches are described, and the README now states for `data/cache/` what the welcome screen promises: a build started from the app writes nothing there. Writing it surfaced **D-75**. The existing sentence "deleting `data/cache/` is always safe" was false on arrival. It was added on 29 September (`a2cd0a8`), two days after the classification cache was un-ignored, and the three sweep outputs had been tracked since July.
- **The API key section**: confirmed against D-68. The app reads the key only from the environment, `streamlit run` does not read `.env`, and on Community Cloud the key goes at the **root level** of Secrets, not under a `[section]`.

Step 3 — Real-data sweep (done, 6 October)

Nothing real ships. Most of this is already settled; this step is the confirmation, not the work.

Item	State
`data/raw/` gitignored	confirmed (`.gitignore:8`); `data/raw` is in no commit on any ref
`.streamlit/secrets.toml` gitignored	done in D-68
Classification cache synthetic-only	done — the six `title N||` test entries were removed and the cache now holds only the synthetic titles
`data/spend/daily.json` gitignored	done in the free-tier slice
No real export anywhere in the tree	confirmed; the only `Connections.csv` ever committed is `data/synthetic/`

A full-history scan is worth doing here rather than trusting the current working tree, since step 4 is the last moment it can matter.

Result, 6 October: done, with two findings. The table's five rows held. The findings came from asking what is in the index and what is in the text, rather than what the rules exclude: - **D-76.** Listing the index (`git ls-files data/`) found 13 tracked files under `data/`, five of them derived from the real network. All were added before the ignore rules, and `.gitignore` never untracks anything. - `cluster_top_terms.png` (the cluster labels, which are real job titles, drawn as an image) is untracked. - `graph_metrics.csv` is untracked as well: a Phase 1 output of the discarded HDBSCAN pipeline, which no longer stands as evidence for anything the project asserts. - The three Phase 1 sweep outputs stay, as evidence for the UMAP parameters and for dropping HDBSCAN. - 12 files are now tracked under `data/`. - **D-78.** The full-history scan (441 real full names, every blob on all 155 commits) found five real connections named in prose in `docs/project_state_decision_log.md`. They are redacted in place, in the `.md` and the `.pdf`. A surname-only pass at HEAD found nothing more. A mention by first name alone or by an unrelated nickname is beyond both scans. - **D-77**, found while tracing D-76's figures: two Phase 1 methodology records still read as current. They need dated notes before the repo is public. It is open.

Every earlier real-data check looked where real data is supposed to live. Both findings were somewhere else: in the index, and in prose.

Step 4 — Fresh public repo (done, 7 October)

Option 2, agreed on 22 September: a **new public repository, single commit, no history**. The current repo stays private as the working archive.

This is the irreversible step, and it carries a decision that is worth taking deliberately rather than by default. See §3 below: decided 6 October, option 2 with the middle path.

Result. The public repository is `amrxshariff/avian_v1`, one commit. The first cut was made by hand at 00:07 on 7 October, from private `0f2dd41`. A filesystem copy on the first attempt brought `.env` and `data/raw/Connections.csv` across, and only `.gitignore` kept them out of the commit. So cuts are now made by `tools/cut_public.ps1`, which builds from `git archive HEAD` and refuses rather than repairs.

The first scripted cut (`311b4c7`) force-pushed over a title edit made in GitHub's web editor on the public side, because every check compared against a stale local copy (**D-82**). The edit was restored in the private repo. The script now asks the remote for its `main`, refuses unless the local copy matches, and leases the push on it. The current public commit is `f0bd4e1`.

Step 5 — Community Cloud (done, 7 October)

- **Python 3.11** in Advanced settings — not the default
- `ANTHROPIC_API_KEY` in Secrets, **at root level**, not nested under a section (D-68 pinned the nested case as not-found)

Result. Deployed at `avianv1.streamlit.app` on Python 3.11, with the key at root level. Gate check 2 verified the root-level secret: an upload with no key pasted was classified on the project key, which closes the question D-68 left open about Community Cloud. The deploy ran with no app-level spend cap until D-81 and D-80 were fixed and re-cut. Until then, the Console's monthly limit was the only cap.

Step 6 — Silence the file-watcher noise (done, 7 October)

Log noise only; no behaviour change.

Result. `.streamlit/config.toml` sets `fileWatcherType = "none"` (`c39a2ec`). It is committed in the private repo and reaches the public one through the cut. The periodic "Examining the path of torch.classes" lines stopped. Four occasional lines remained in the two hours after the redeploy, and their source is not established. In Streamlit 1.41 the only emitter is the source watcher, which is not constructed under `"none"`. So they came from a process without the new config, but which one is unverified (gate record, step 6's qualification). **If those lines reappear after a cold start on the current config, that explanation is wrong and this needs looking at again.** One trade-off: the setting applies locally too, so `streamlit run app.py` no longer reloads on save. The file says how to get it back.

Step 7 — The gate (done, 7 October)

One continuous session on the deployed app, written verdict per step, no restart — the same discipline as Gates A to D.

#	Check	Passes when	Verdict, 7 October
1	First run, no key, no upload	Lands on the welcome screen, with the demo offered below	**Pass**
2	Upload a small export **without** pasting a key	Shows the **classified** note, not the keyless one — this is the D-68 check that the project key was found	**Pass**: classified, and the provenance line named the project key
3	Grey nodes	Present, and correctable	**Pass**

#	Check	Passes when	Verdict, 7 October
4	Provenance line	Shown on screen for both the demo and an uploaded build	**Pass**: both lines recorded
5	Whole session	Zero crashes	**Pass**

5 of 5. The record is `docs/p2_9_gate_record.md`. Row 1 first read "Lands on the demo". The designed first run is the welcome screen with the demo offered, so the pass is recorded against what happens.

Check 2 is the one that cannot be verified any other way. D-68 closed on evidence covering `streamlit run` and the secrets file it reads — Community Cloud itself is unverified until this runs.

Step 8 — Demo recording

Last, because it should record the deployed app rather than a local one — and because D-70's measurement may change whether the enrichment comparison panel is demonstrated at all.

Decided, 9 October: - **What is recorded.** The deployed app at the cut `21e5500` (P2.10 items 1 to 4), with the project key in Secrets. - **The path.** Upload `data/synthetic/Connections.csv` through the upload panel. This is real code with no real names, and it shows the provenance line, the note "Classified using this demo's shared allowance.", and the chat's "N free questions left". - **A cold run.** A session build starts with an empty cache (D-71), so every title is sent. Use a fresh private window per take, or the session cache makes a second take instant. Each take is a fresh classification run, so the needs-review count will probably differ from the demo's 124 (D-69). A caption makes that point rather than leaving it to look like an inconsistency. - **The wait is shown uncut**, with a caption such as "about N minutes, shown in full". The build has no progress indicator (D-72), and showing the wait honestly is better than a cut that implies it is instant. It also makes D-72's case without anyone having to argue it. - **Scenes:** the demo landing, the upload, grey nodes and a correction, the chat with its allowance caption, and the provenance line. The profile panel is optional: since D-84 it shows one reading with its caveat, which is honest but modest.

Open: which network goes on camera, decided before filming, not during it. - The 442-person synthetic file is the honest demonstration. Its 384 distinct titles go in about 20 batches, and at the rate measured on the deploy for D-72 (about 0.6 s per person) the uncut wait is roughly four and a half minutes. It costs about $0.40 a take. - The 60-person synthetic file has 53 titles in 3 batches, a wait of about half a minute, and costs a few cents.

Know the length before starting. The figures are estimates from D-72's single measurement.

Correction note — 10 October 2026 (P2.11). The decisions above are left as written. Three things have changed since: - **When, and which cut.** The recording moves after P2.11 and D-89's fix. The order is: cut and reboot, D-89, P2.11, record, publish. Recording on `21e5500`, or on today's cut, would film a build that P2.11 makes several times faster within days. A recording is local and reaches no visitor until it is published, so the gate is publishing, and publishing waits for P2.11's re-measure on Cloud. - **The open question is decided: a 60-person upload, filmed at its eventual speed.** An upload shows what a visitor actually does. The demo loads a finished table and never builds, so filming it would be untouched by P2.11, but it would show a canned network. The wait stays uncut and captioned, as decided above. - **The estimates are replaced by measurements.** Two keyed builds of the 442-person export on 10 October took 331 s cold and 299 s warm, locally, with batches averaging about 15 s (`docs/p2_11_scope.md`). The 60-person file's 3 batches therefore take about 46 s, not about half a minute, before any cold-start cost. After P2.11 they run side by side, and the slowest of the three sets the time: about 20 to 25 s. That is a projection, until the recording itself measures it.

3. The decision inside step 4

Decided, 6 October 2026: option 2, with the middle path. The public repository is a single commit with no history, and this repository stays private as the archive. The register and the design records ship at

full fidelity. The README's "How this was built" section (`731a3c4`) says the history is kept privately, and points to `docs/defect_register.md` as the record of it.

The reason is the one below, made concrete by step 3. The private history does not only hold real data under `data/`, which a path-based rewrite could strip: - five real connections named in prose in the decision log, in 101 commits (D-78) - real job titles drawn in an image (D-76)

A safe rewrite would have to scrub the contents of documents, not just paths, and would still rest on scans whose limits D-78 records. Starting clean has no such dependency.

The deliberation that led here is left as written:

A single squashed commit keeps every **file** — the design records, the defect register, the gate records all ship. What it discards is the **sequence**: which fix came from which failure, and the commit messages that explain them.

The last ten days alone produced fifteen defect numbers, five design records, and a register that exists specifically to explain why things are the way they are. That trail is the strongest evidence of how this was built, and it is the thing a reader assessing the work would find most unusual.

The counter-argument for option 2 has not changed: the private history contains real LinkedIn data and real classifications, and rewriting it safely is more work and more risk than starting clean.

A middle path worth considering before committing: ship the single commit, but include `docs/defect_register.md` and the design records at full fidelity, and add a short README line stating that the development history is maintained privately and the register is the record of it. That keeps the evidence without touching the data problem.

Decide this before step 4, not during it.

4. Open, not blocking

D-70's CLI measurement. Run `compare()` a handful of times on one real title and About through the CLI, to separate this path's own noise from the prompt confound.

Roughly ten minutes and a few cents, and it distinguishes two quite different outcomes:

- **Stable per prompt, consistently different between them** → the confound is the prompt alone, the fix is prompt work, and the feature is recoverable
- **Each reading unstable in itself** → the follow-up is asking each reading two or three times and naming a cause only when both sides are unanimous and disagree. That is 3× the calls, charged to someone's key, so it should rest on a measurement rather than an assumption

It does not block the deploy. It should happen **before step 8**, because the answer may change whether that panel appears in the recording at all. The panel currently ships with an honest caveat saying a difference may not come from the description — enough for launch, but it is a caveat admitting the feature cannot do what it appears to do, sitting on the screen of a deployed app.

Cosmetic, when next in the file: `retry_offer.py:92` could take a `{'' if n == 1 else 's'}` plural, as `chat_allowance.py:49` already does. No stutter risk today — `take_offer` returns early on an empty target list and `MIN_PEOPLE` stops a one-person network reaching a build at all.

5. After P2.9

1. **Phase 2 checkpoint** — formally closed by a deployed app that renders a real network and lets someone correct a grey node
2. **API**
3. **MCP server.** Note the constraint found on 17 September: the MCP *sampling* mechanism this would naturally rely on was deprecated in the 2026-07-28 protocol revision. The proposed alternative is a tool-call round trip for classification instead

4. **Hosting**

No dates. Day counting, end-dates and baseline comparison were retired on 22 September; sequencing and dependency order are the only schedule this project keeps.